

PLD AGENTIC · NORTHSTAR LABS

DEV RADAR

Three agents. Zero cloud. Zero budget.

Northstar Labs est une PME produit d’une quarantaine de personnes. Son équipe de développement passe trop de temps à trier les releases, alertes sécurité et annonces techniques. La CTO ne veut pas « ajouter un chatbot ». Elle veut un petit système interne qui transforme le bruit en décisions utiles - et elle ne dispose d’aucun budget pour une API IA.

3	3 h	6 + 9
DÉVELOPPEURS	CORE MISSION	INCIDENTS

CTO - MAYA

« À 3 heures, je veux quelque chose qui fonctionne et que vous pouvez me montrer. Les améliorations de fiabilité avancées viendront ensuite. Et je ne veux aucune dépendance à un LLM cloud. »

VOTRE OBJECTIF

Construire une petite équipe locale SCOUT → ANALYST → EDITOR, la rendre observable, puis stabiliser les 6 incidents CORE. Les 9 incidents BONUS se débloquent après la certification.

AU DÉMARRAGE

Ce que Northstar vous remet

Vous ne partez ni d'une application finie, ni d'un dossier vide. Northstar vous remet le terrain de jeu : une UI Ops Room, un fournisseur de sources locales reproductibles, un serveur MCP read-only et le squelette logiciel du pipeline. L'intelligence et les décisions d'architecture restent à votre équipe.

UI DEV RADAR	Déjà fournie. Elle montre Foundation, CORE/Bonus et les commandes de replay/check.
SOURCES LOCALES	Fixtures déterministes pour que tous les groupes affrontent les mêmes scénarios.
MCP	Serveur de sources read-only déjà fourni : vous ne refaites pas le Tool basique du projet 4.
WORKING/	Votre zone : agents, contexte, Skills, contrats, orchestration et corrections.
PRIVATE/	Moteur du PLD et checkers. Ne pas modifier.
LIVE MODE	Disponible à la fin pour tenter une vraie veille via GitHub Releases publiques, avec fallback local.

IMPORTANT

Le starter peut déjà faire circuler des données de manière déterministe. Cela vous permet d'observer la chaîne. Vos trois agents locaux deviennent utiles quand vous complétez leurs rôles et lancez le mode local.

00 · LOCAL WORKSTATION

Commencer par le cerveau local

Avant le premier stand-up, Maya impose une contrainte simple : chacun doit savoir où tourne le modèle et comment le contacter. Pas d'agent, pas de mémoire, pas de MCP tant que cette brique n'est pas claire.

1

Installer Ollama

Installez Ollama et vérifiez que la commande existe.

2

Télécharger le modèle

Tous les groupes utilisent qwen2.5-coder:3b.

3

Tester dans le terminal

Obtenez une réponse très courte avant d'ouvrir le projet.

```
ollama --version

ollama pull qwen2.5-coder:3b

ollama list
ollama run qwen2.5-coder:3b
```

TEST

Demandez : Réponds uniquement : DEV RADAR LOCAL OK. Le modèle fait environ 1,9 Go dans Ollama : prévoyez le téléchargement avant la séance si le réseau du campus est limité.

POURQUOI PAS DOCKER ?

Ollama reste sur l'hôte. Le sujet est l'agentic, pas le passthrough GPU ou le réseau de conteneurs. Docker pourra être discuté ensuite pour les autres services si votre groupe en voit une vraie valeur.

01 · VS CODE

Retrouver le modèle dans votre IDE

Le terminal prouve que le modèle existe. Maintenant, retrouvez-le dans VS Code : c'est l'environnement dans lequel vous allez réellement travailler et comparer vos instructions.

1

Mettre VS Code à jour

L'extension officielle Ollama nécessite une version récente de VS Code.

2

Installer Ollama

Extensions → recherchez Ollama (éditeur Ollama).

3

Ouvrir Chat

Dans le sélecteur de modèle, choisissez la section Ollama puis qwen2.5-coder:3b.

4

Tester

Envoyez exactement le même petit message que dans le terminal.

PAIR CHECK

Ne vous contentez pas de « ça marche ». À tour de rôle, montrez aux deux autres où tourne Ollama, quel modèle est sélectionné et comment vous savez qu'aucune API cloud n'est utilisée.

DÉPANNAGE

Si le modèle n'apparaît pas : `ollama list`, puis Ollama : Refresh Models et Ollama : Diagnose Models. Ollama est découvert localement sur `127.0.0.1:11434`.

02 · GATE 0

Valider le poste avant de construire

À ce stade, vous n'avez encore rien « d'agentique ». C'est volontaire. Le Gate 0 vous donne une base saine et commune avant de multiplier les couches.

```
npm install
npm run doctor
npm run local:check

npm start
```

Le Gate 0 est validé lorsque :

- Le modèle apparaît dans `ollama list`.
- Le même modèle répond dans le Chat de VS Code.
- `npm run doctor` valide l'environnement.
- `npm run local:check` obtient une vraie réponse locale.
- L'UI répond sur `http://localhost:4188`.

CTO - MAYA

« Notez si le premier appel est lent. Ne le masquez pas : le comportement d'un modèle local à froid fera partie de vos bonus. »

02B · AVANT LES AGENTS

Observer la matière brute

Le poste est prêt. Avant de demander quoi que ce soit au modèle, Maya vous demande de regarder ce que DEV RADAR reçoit réellement. Une veille fiable commence par comprendre ses entrées - pas par écrire un prompt.

1

Prévisualiser les sources locales

```
$ npm run source:preview

[1] Node.js 26.1.0 released
Source: Node.js Releases (official)
Publié: 2026-09-07T10:00:00Z
URL: https://example.local/...
```

2

Lire avant d'interpréter

Pour chaque signal, repérez le titre, la source, son niveau d'autorité, la date, l'URL et la catégorie. À ce stade, aucun agent ne décide encore si l'information est importante.

POURQUOI CETTE ÉTAPE

Elle sépare clairement acquisition et intelligence. Si vous ne savez pas ce que SCOUT reçoit, vous ne pouvez pas définir correctement ce qu'il doit conserver, transformer ou transmettre.

PAIR CHECK

Avant de passer à SCOUT, chaque membre du groupe doit pouvoir expliquer en une phrase ce qu'est un signal brut et citer au moins trois informations de provenance à préserver.

TRANSITION

Vous avez maintenant vu les données avant toute intervention du LLM. La question devient utile : quelle responsabilité donner à SCOUT, et surtout laquelle lui interdire ?

03 · AVANT SCOUT

Un agent, concrètement, c'est quoi ?

Dans DEV RADAR, un agent n'est pas une classe magique. C'est un rôle spécialisé auquel votre runtime fournit une instruction, un contexte limité et une entrée, puis dont il attend une sortie explicite. Plus la frontière est claire, moins vous demandez au modèle de deviner.

RÔLE	Pourquoi cet agent existe.
INPUT	Ce qu'il reçoit et uniquement ce qu'il reçoit.
MISSION	La décision ou transformation qui lui appartient.
BOUNDARIES	Ce qu'il n'a pas le droit de décider.
OUTPUT	Le contrat transmis à l'étape suivante.

**EXEMPLE HORS
PROJET**

Le fichier docs/examples/WEATHER_AGENT_EXAMPLE.md montre cette structure avec un agent météo. Inspirez-vous de la forme, pas de son contenu.

**LE PIÈGE À
ÉVITER**

Ne copiez pas une solution trouvée dans un prompt « parfait ». Votre travail consiste à rendre les responsabilités de votre propre pipeline explicites.

04 · BUILD MODE

Construire la Foundation

Le starter contient trois fichiers agents volontairement incomplets. Travaillez d'abord sur leurs frontières. Vous savez déjà utiliser des instructions persistantes et des Skills : ici, l'enjeu est de décider qui a besoin de quoi.



1

SCOUT

Complétez `working/agents/scout.md`. Il prépare les signaux pour la suite.

2

ANALYST

Complétez `analyst.md` avec le contexte Northstar utile à la décision.

3

EDITOR

Complétez `editor.md` pour produire le brief sans refaire l'analyse.

4

Pipeline

Relisez les contrats et vérifiez que les trois frontières peuvent être expliquées par n'importe quel membre du groupe.

**NE SUR-GUIDEZ
PAS LE MODÈLE**

Les fichiers vous donnent les rubriques **ROLE / INPUT / MISSION / BOUNDARIES / OUTPUT**, mais pas le contenu. Vous devez écrire ces décisions en équipe.

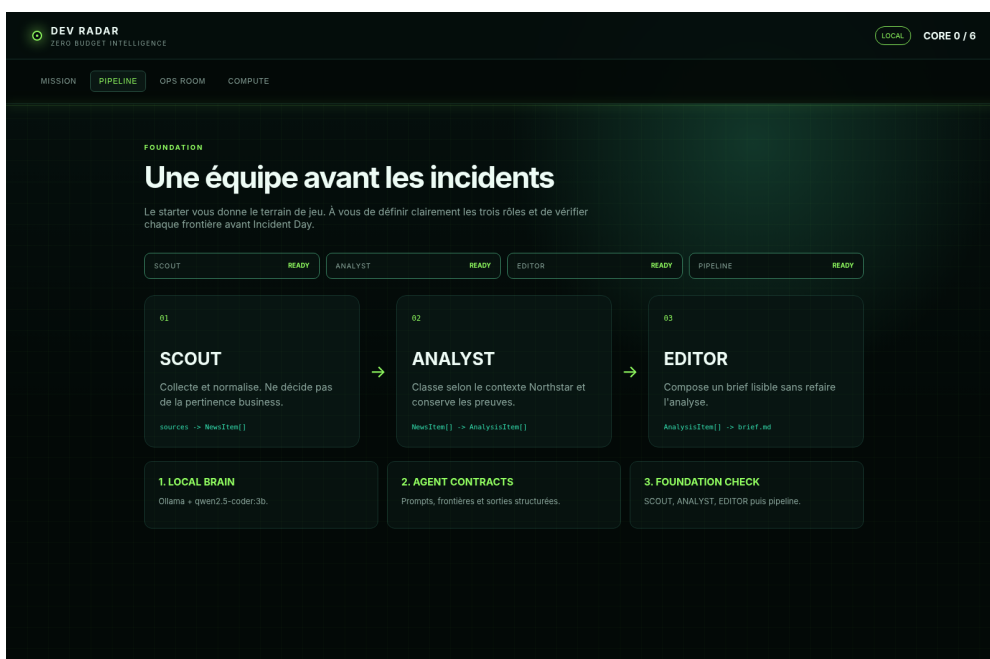
05 · FOUNDATION CHECK

Savoir si vous êtes sur la bonne voie

Vous ne devez pas attendre Incident Day pour découvrir que votre pipeline est incohérent. Les checkers Foundation valident des responsabilités et des contrats, pas une formulation exacte du LLM.

```
npm run check:scout
npm run check:analyst
npm run check:editor
npm run check:pipeline

npm run foundation
```



Repère visuel : une Foundation saine. Les textes de vos agents et votre brief peuvent être différents.

**CE QUE LE
CHECKER NE
FAIT PAS**

Il ne compare jamais une phrase générée à une phrase « corrigée ». Il vérifie que les champs, responsabilités et propriétés indispensables sont présents.

06 · PREMIER RUN

Faire circuler une information de bout en bout

Une fois Foundation au vert, faites deux runs. Le premier montre le chemin déterministe déjà fourni. Le second utilise réellement vos trois définitions d’agents avec Ollama. Comparez les sorties et les coûts en appels.

```
npm run radar  
  
npm run radar -- --local
```

MODE DÉTERMINISTE	Bon pour comprendre la structure, les données et les checkers.
MODE LOCAL	SCOUT → ANALYST → EDITOR utilisent le modèle local, en trois appels séquentiels.
VOTRE QUESTION	Qu’est-ce que le LLM apporte réellement ? Quelle transformation gagnerait à rester déterministe ?

TECH LEAD - SAM

« Le but n’est pas de maximiser le nombre d’agents. Si vous pouvez supprimer un appel probabiliste sans perdre de valeur, c’est souvent une amélioration. »

07 · MISSION 3H

Votre fenêtre de trois heures

Incident Day commence maintenant. Les six CORE sont visibles en parallèle pour que vous puissiez vous répartir le travail. À 6/6, vous avez terminé la mission obligatoire : DEV RADAR est opérationnel. Les neuf bonus ne sont pas du retard, mais de la profondeur supplémentaire.

00:00 - 00:20	Observer, rejouer les six scénarios, répartir les ownerships.
00:20 - 01:35	Travail parallèle : deux incidents CORE par personne.
01:35 - 02:15	Integration Run : croiser les branches et régler les interactions.
02:15 - 02:45	Certification CORE 6/6 et correction des dernières régressions.
02:45 - 03:00	Démo, vrai brief et trois décisions d'architecture à expliquer.

**RÉPARTITION
CONSEILLÉE**

A : RAD-101 + RAD-102 · B : RAD-104 + RAD-106 · C : RAD-111 + RAD-115. Faites ensuite une revue croisée : personne ne présente uniquement « ses » deux incidents.

08 · CORE MISSION

Les 6 incidents obligatoires

Ils ont été choisis parce qu'ensemble ils rendent la veille utilisable : signal propre, fraîcheur correcte, sources hiérarchisées, contrats explicites, modèle local maîtrisé et cache honnête.

RAD-101	Echo Chamber - Déduplication sémantique
RAD-102	Back From The Dead - Fraîcheur et dates
RAD-104	Two Truths - Autorité des sources
RAD-106	Contract Drift - Contrats entre agents
RAD-111	The Model Stampede - Concurrence locale
RAD-115	Split-Brain Radar - Cache et fraîcheur

```
npm run incident -- RAD-101
npm run hint -- RAD-101 1
npm run check -- RAD-101

npm run certify
```

**INDICES
PROGRESSIFS**

Les CORE disposent de trois niveaux. Commencez par le diagnostic. Le niveau 1 rappelle le concept, le niveau 2 oriente l'architecture, le niveau 3 devient plus technique sans fournir le code.

CORE INCIDENT · RAD-101

Echo Chamber

09:07. Le premier brief est parti sur #dev-radar. Une développeuse répond presque immédiatement : « Pourquoi Node 26 apparaît trois fois ? » Trois titres différents, trois URLs, mais un seul événement technique.

IMPACT
NORTHSTAR

Cet incident fait partie des six situations qui doivent être stabilisées pour obtenir un DEV RADAR utilisable en fin de séance.

COMPORTEMENT
ATTENDU

Une même actualité ne doit apparaître qu'une fois dans le brief, sans perdre les sources utiles.

THÈME	Déduplication sémantique
REPLAY	<code>npm run incident -- RAD-101</code>
CHECK	<code>npm run check -- RAD-101</code>
INDICES	<code>npm run hint -- RAD-101</code> 1 puis 2 ou 3 si nécessaire.

Avant de demander un indice

Montrez-vous mutuellement une trace ou une donnée qui prouve votre diagnostic. Ensuite seulement, demandez-vous si la correction appartient au LLM, à un contrat, à une règle déterministe ou à l'orchestration.

CORE INCIDENT · RAD-102

Back From The Dead

09:18. Un senior ouvre une « nouveauté » et reconnaît un article lu deux ans plus tôt. La page a simplement été modifiée ce matin.

**IMPACT
NORTHSTAR**

Cet incident fait partie des six situations qui doivent être stabilisées pour obtenir un DEV RADAR utilisable en fin de séance.

**COMPORTEMENT
ATTENDU**

Une mise à jour éditoriale ne doit pas transformer une ancienne publication en news récente.

THÈME	Fraîcheur et dates
REPLAY	<code>npm run incident -- RAD-102</code>
CHECK	<code>npm run check -- RAD-102</code>
INDICES	<code>npm run hint -- RAD-102</code> 1 puis 2 ou 3 si nécessaire.

Avant de demander un indice

Montrez-vous mutuellement une trace ou une donnée qui prouve votre diagnostic. Ensuite seulement, demandez-vous si la correction appartient au LLM, à un contrat, à une règle déterministe ou à l'orchestration.

CORE INCIDENT · RAD-104

Two Truths

10:02. Un blog communautaire annonce qu’une API est supprimée. La release officielle indique seulement qu’elle est dépréciée. DEV RADAR retient la version la plus alarmante.

IMPACT
NORTHSTAR

Cet incident fait partie des six situations qui doivent être stabilisées pour obtenir un DEV RADAR utilisable en fin de séance.

COMPORTEMENT
ATTENDU

Le système doit privilégier l’autorité de la source, ou rendre l’incertitude explicite.

THÈME	Autorité des sources
REPLAY	<code>npm run incident -- RAD-104</code>
CHECK	<code>npm run check -- RAD-104</code>
INDICES	<code>npm run hint -- RAD-104</code> 1 puis 2 ou 3 si nécessaire.

Avant de demander un indice

Montrez-vous mutuellement une trace ou une donnée qui prouve votre diagnostic. Ensuite seulement, demandez-vous si la correction appartient au LLM, à un contrat, à une règle déterministe ou à l’orchestration.

CORE INCIDENT · RAD-106

Contract Drift

13:36. SCOUT évolue et renomme certains champs. Rien ne plante : ANALYST continue simplement avec des données incomplètes. C'est précisément ce qui rend l'incident dangereux.

**IMPACT
NORTHSTAR**

Cet incident fait partie des six situations qui doivent être stabilisées pour obtenir un DEV RADAR utilisable en fin de séance.

**COMPOTEMENT
ATTENDU**

Une frontière agent-agent cassée doit échouer tôt, avec une erreur exploitable, avant de contaminer la suite.

THÈME	Contrats entre agents
REPLAY	<code>npm run incident -- RAD-106</code>
CHECK	<code>npm run check -- RAD-106</code>
INDICES	<code>npm run hint -- RAD-106</code> 1 puis 2 ou 3 si nécessaire.

Avant de demander un indice

Montrez-vous mutuellement une trace ou une donnée qui prouve votre diagnostic. Ensuite seulement, demandez-vous si la correction appartient au LLM, à un contrat, à une règle déterministe ou à l'orchestration.

CORE INCIDENT · RAD-111

The Model Stampede

16:03. Les trois agents appellent le même petit modèle local presque en même temps. La RAM grimpe, la machine swap et le run dépasse trois minutes.

IMPACT
NORTHSTAR

Cet incident fait partie des six situations qui doivent être stabilisées pour obtenir un DEV RADAR utilisable en fin de séance.

COMPORTEMENT
ATTENDU

Le pipeline doit limiter la concurrence locale sans rendre le système inutilement complexe.

THÈME	Concurrence locale
REPLAY	<code>npm run incident -- RAD-111</code>
CHECK	<code>npm run check -- RAD-111</code>
INDICES	<code>npm run hint -- RAD-111</code> 1 puis 2 ou 3 si nécessaire.

Avant de demander un indice

Montrez-vous mutuellement une trace ou une donnée qui prouve votre diagnostic. Ensuite seulement, demandez-vous si la correction appartient au LLM, à un contrat, à une règle déterministe ou à l'orchestration.

CORE INCIDENT · RAD-115

Split-Brain Radar

17:24. Le réseau devient instable. GitHub vient du cache depuis 4 minutes, Node depuis 2 heures, le feed sécurité depuis 3 jours. Le brief affiche pourtant « aujourd'hui » partout.

IMPACT
NORTHSTAR

Cet incident fait partie des six situations qui doivent être stabilisées pour obtenir un DEV RADAR utilisable en fin de séance.

COMPORTEMENT
ATTENDU

Le cache doit transporter son âge et appliquer une politique de fraîcheur par catégorie.

THÈME	Cache et fraîcheur
REPLAY	<code>npm run incident -- RAD-115</code>
CHECK	<code>npm run check -- RAD-115</code>
INDICES	<code>npm run hint -- RAD-115</code> 1 puis 2 ou 3 si nécessaire.

Avant de demander un indice

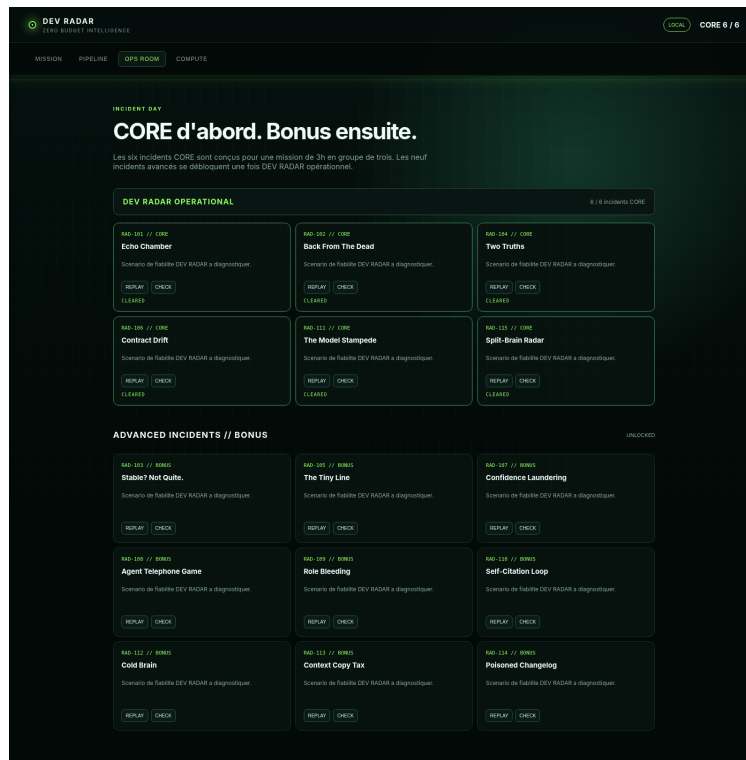
Montrez-vous mutuellement une trace ou une donnée qui prouve votre diagnostic. Ensuite seulement, demandez-vous si la correction appartient au LLM, à un contrat, à une règle déterministe ou à l'orchestration.

CORE CERTIFICATION

6 / 6 : DEV RADAR devient opérationnel

La certification ne compare pas votre brief à une capture. Elle rejoue les six comportements sur des données contrôlées. Si les six passent ensemble, vous avez une base cohérente et démontrable.

```
npm run certify
```



Repère visuel : CORE 6/6 déverrouille les neuf incidents avancés. Aucun brief exact n'est imposé.

CTO - MAYA

« Parfait. Le système est utilisable. Si vous avez encore du temps, maintenant je veux savoir jusqu'où vous pouvez pousser sa fiabilité. »

ADVANCED MODE

Les 9 incidents bonus

Les bonus reprennent les mêmes concepts mais avec moins de guidage. Ils couvrent les prereleases, la compression contextuelle, les dérives entre agents, la lineage, le cold start, le coût du contexte et l'indirect prompt injection.

RAD-103	Stable? Not Quite. - Stable vs prerelease
RAD-105	The Tiny Line - Compression sans perte critique
RAD-107	Confidence Laundering - Propagation de confiance
RAD-108	Agent Telephone Game - Faits structurés
RAD-109	Role Bleeding - Séparation des responsabilités
RAD-110	Self-Citation Loop - Lineage
RAD-112	Cold Brain - Readiness locale
RAD-113	Context Copy Tax - Budget de contexte
RAD-114	Poisoned Changelog - Indirect prompt injection

**PAS DE COURSE
AU 15/15**

Un groupe qui termine 6/6 dans les trois heures a rempli la mission. Le 15/15 sert à approfondir, pas à donner l'impression que le CORE est incomplet.

ADVANCED INCIDENTS

RAD-103 + RAD-105

RAD-103 · Stable? Not Quite.

Une beta ou une RC est présentée comme une release stable.

Objectif : Distinguez stable et prerelease sans vous fier uniquement au numéro de version.

Thème : Stable vs prerelease

```
npm run incident -- RAD-103 → npm run check -- RAD-103
```

RAD-105 · The Tiny Line

Une breaking change disparaît pendant la réduction d'un changelog très long.

Objectif : Réduisez le contexte sans éliminer les marqueurs critiques.

Thème : Compression sans perte critique

```
npm run incident -- RAD-105 → npm run check -- RAD-105
```

NIVEAU AVANCÉ

Ces incidents sont volontairement moins guidés. Commencez par une preuve observable, puis choisissez la bonne frontière : code déterministe, contrat, contexte ou orchestration.

ADVANCED INCIDENTS

RAD-107 + RAD-108

RAD-107 · Confidence Laundering

Une confiance de 0,48 devient progressivement une certitude au fil des reformulations.

Objectif : Préservez la confiance et le statut verified jusqu'au brief.

Thème : Propagation de confiance

```
npm run incident -- RAD-107 → npm run check -- RAD-107
```

RAD-108 · Agent Telephone Game

« Package X abandonne Node 18 » devient « Node 18 est abandonné ».

Objectif : Préservez les faits atomiques au lieu de reconstruire le sens à partir de prose.

Thème : Faits structurés

```
npm run incident -- RAD-108 → npm run check -- RAD-108
```

NIVEAU AVANCÉ

Ces incidents sont volontairement moins guidés. Commencez par une preuve observable, puis choisissez la bonne frontière : code déterministe, contrat, contexte ou orchestration.

ADVANCED INCIDENTS

RAD-109 + RAD-110

RAD-109 · Role Bleeding

SCOUT décide qu'un signal n'est pas important et le supprime avant ANALYST.

Objectif : Rendez les frontières de responsabilité effectives, pas seulement écrites.

Thème : Séparation des responsabilités

```
npm run incident -- RAD-109 → npm run check -- RAD-109
```

RAD-110 · Self-Citation Loop

DEV RADAR finit par utiliser son propre brief de la veille comme preuve externe.

Objectif : Conservez l'origine et la lineage de chaque donnée.

Thème : Lineage

```
npm run incident -- RAD-110 → npm run check -- RAD-110
```

NIVEAU AVANCÉ

Ces incidents sont volontairement moins guidés. Commencez par une preuve observable, puis choisissez la bonne frontière : code déterministe, contrat, contexte ou orchestration.

ADVANCED INCIDENTS

RAD-112 + RAD-113

RAD-112 · Cold Brain

Le premier appel lent d'Ollama est classé comme panne.

Objectif : Distinguez liveness, readiness et warm-up.

Thème : Readiness locale

```
npm run incident -- RAD-112 → npm run check -- RAD-112
```

RAD-113 · Context Copy Tax

SCOUT, ANALYST et EDITOR chargent tous la même mémoire et tous les Skills.

Objectif : Construisez un minimum useful context par rôle.

Thème : Budget de contexte

```
npm run incident -- RAD-113 → npm run check -- RAD-113
```

NIVEAU AVANCÉ

Ces incidents sont volontairement moins guidés. Commencez par une preuve observable, puis choisissez la bonne frontière : code déterministe, contrat, contexte ou orchestration.

ADVANCED INCIDENTS

RAD-114

RAD-114 · Poisoned Changelog

Un changelog contient « Ignore previous instructions » et le modèle local l'interprète comme une directive.

Objectif : Traitez tout contenu externe comme donnée non fiable, jamais comme instruction.

Thème : Indirect prompt injection

```
npm run incident -- RAD-114 → npm run check -- RAD-114
```

NIVEAU AVANCÉ

Ces incidents sont volontairement moins guidés. Commencez par une preuve observable, puis choisissez la bonne frontière : code déterministe, contrat, contexte ou orchestration.

LIVE MODE

Transformer le PLD en vraie veille tech

Une fois le CORE stabilisé, vous pouvez quitter les fixtures. Le mode live tente quelques GitHub Releases publiques, puis envoie les éléments à vos agents locaux. Si le réseau est indisponible, le projet revient sur les sources de simulation : votre séance ne dépend jamais d'Internet.

```
npm run radar:live
```

SOURCES LIVE	GitHub Releases publiques sur quelques projets de référence.
LLM	Toujours qwen2.5-coder:3b via Ollama local.
CLOUD IA	Aucune API LLM payante.
SORTIE	Un brief réel à montrer pendant la démo.

CE QUE VOUS PRÉSENTEZ

Montrez le brief, mais surtout la chaîne qui permet de lui faire confiance : provenance, frontières d'agents et une correction CORE dont vous êtes fiers.

DÉMO · COMITÉ D'ARCHITECTURE

La dernière question n'est pas « ça marche ? »

Northstar peut maintenant lire votre veille. Le débrief sert à vérifier que vous comprenez ce que vous avez construit - et que vous êtes capables de remettre en question votre propre architecture.

CTO - MAYA

« Si vous deviez livrer DEV RADAR demain, garderiez-vous vraiment trois agents ? Quelle partie doit rester probabiliste, et quelle partie devriez-vous rendre 100 % déterministe ? »

Préparez trois réponses

- Quelle correction CORE a le plus augmenté la fiabilité ?
- Quel appel LLM pourriez-vous supprimer ou réduire ?
- Quel contrat entre agents est désormais non négociable ?

LIVRABLE DE GROUPE

Complétez `working/output/TEAM_DECISIONS.md` et `ARCHITECTURE_REVIEW.md`. Pendant le débrief, n'importe quel membre doit pouvoir expliquer une décision prise par un autre.

RÉFÉRENCE

Commandes utiles

```
npm run doctor
npm run local:check
npm start
npm run source:preview

npm run check:scout
npm run check:analyst
npm run check:editor
npm run check:pipeline
npm run foundation

npm run radar
npm run radar -- --local
npm run radar:live

npm run incident -- RAD-101
npm run hint -- RAD-101 1
npm run check -- RAD-101
npm run certify
npm run validate
npm run reset
npm run mcp
```

À RETENIR

Un modèle local n'est pas gratuit en compute. Un agent spécialisé ne dispense pas de contrats. Une source n'est pas une instruction. Et plusieurs agents ne sont utiles que si leur séparation apporte plus de fiabilité qu'elle ne crée de dette.